

COMPUTER SCIENCE TRIPOS Part IB – 2021 – Paper 4

1 Programming in C and C++ (djg11)

declarations and
definitions

- (a) (i) In programming languages in general, what is the difference between statically and dynamically-allocated variables? Without using the `static` keyword, give examples of both types of variable in the C language. [2 marks]
- (ii) What effect do the `static` and `extern` keywords have in C for both forms of variable? [3 marks]

Answer: Dynamic variables are allocated storage at run-time and their number may vary from one run to another. Static variables do not vary in quantity and can be allocated an address at compile time. In C, a declaration at the global level such as `int s=0;` defines a static variable, whereas the formal parameters and locals (automatics) to functions are dynamic. All heap data is dynamic storage too.

The `static` modifier at the global level has the (perhaps misnamed) effect of hiding the definition from the linking process so it is not visible to a separate compilation unit. On the other hand, the `extern` declaration enables a program to compile and use the declared item, but relies on it being defined/implemented either in the same compilation unit or else in another one supplied during linking.

Applied to local variables within a function, the `static` modifier preserves its scope, but moves its definition from the stack to the static data segment, so that values persist from one call to another. Applied to local variables within a function, the `extern` modifier has a similar effect of making the variable static, but does not provide a definition, so it must be defined elsewhere and can be freely accessed from elsewhere.

bit-level
operations

- (b) There are 32 characters in the ASCII set between ‘A’ and ‘a’. All are printable. A potentially-infinite stream of 8-bit bytes can be made printable by encoding into base 32 and represented by a longer stream just using these characters. Give a syntactically-accurate definition in C for `encode32(unsigned char a)`, called for each byte, which invokes `putchar(char c)` once or twice per call to render the encoded output. [5 marks]

Answer:

I was going to ask about base64, but open book allowed code from online to be pasted in.

```
void encode32(unsigned char a)
{
    static int bits = 0; static unsigned int d = 0;
    d |= (a<<bits); bits+=8;
    while (bits>=5) { putchar('A'+(d&0x1F)); d >>= 5; bits -= 5; }
    } // Need to flush with a final call to encode32(0) for a finite stream.
```

Statements

- (c) Show how the `for` statement in C can be used to traverse a linked list. Explain the benefits of this coding style, mentioning whether the `continue` statement works appropriately. [4 marks]

Answer: `for (node *s=head; s; s=s->next)`
`{ blah ...; if (...) continue; blah2 ... }`

The advantages are clarity of intent and, even given complex control flow in the loop body, it is impossible to accidentally bypass the forward chaining operation and start the next iteration on the same list item. Yes, `continue` statement works properly, advancing to the next list item regardless of surrounding `if` blocks.

Memory
organisation

```
typedef char * mystring;  
mystring s201 = "201";  
mystring s202 = s201;  
s202[2] += 1;
```

A novice programmer writes the above code. What do you think they are intending to do and what two problems might they suffer? [2 marks]

Answer: They appear to want two strings to containing the numbers 201 and 202 as text. The typedef is a red herring. The second string will be an alias for the first, so changing it also changes the first one. The string "201" is likely to be stored in a read-only memory segment and so the write to it will fault. Note: `std::string` in C++ overcomes these issues.

Memory
organisation

- (e) An interpreter for a string processing language is written in C. Describe four storage and efficiency-related considerations when designing the module for storing strings for the interpreter? [4 marks]

Answer: There will be lots of strings so it needs to be efficient. For long running programs, many strings are likely to have relatively short lifetimes and so garbage collection will be needed. String comparison may be commonplace, so storing each string once on a hashing basis would provide efficiency through pointer comparison. Strings are likely to have all sorts of lengths and fragmentation of store will likely arise. Using the library `malloc` and `free` could well be efficient enough, as it will have the necessary facilities to avoid fragmentation (buddy schemes etc.). String catenation is likely to be common, but copying long strings is hideously expensive, so storing them as a tree of catenated fragments may be a good approach. *Any correct four points for four marks.*
